

LAPORAN PRAKTIKUM STRUKTUR DATA

SINGLE LINKED LIST

disusun oleh :

Khairunnisa M.

NIM 2511532005

Dosen Pengampu :

Dr. Wahyudi,. S.T., M.T.

Asisten Dosen :

M. Ghalid



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji Syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa atas rahmat dan karunia-Nya laporan praktikum Struktur Data dengan materi Single Linked List ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bentuk dokumentasi dan pemahaman penulis terhadap konsep Single Linked List dalam struktur data serta implementasinya menggunakan bahasa pemrograman Java.

Dalam laporan ini, penulis membahas beberapa implementasi Single Linked List, serta operasi – operasi dasar yang terdapat pada Single Linked List.

Penulis menyadari bahwa lapoaran ini masih memiliki kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun untuk perbaikan di masa mendatang. Akhir kata, semeoga laporan ini dapat memberikan manfaat bagi pembaca.

Padang, 04 Mei 2026

Khairunnisa M.

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I.....	1
PENDAHULUAN	1
1.1. Latar Belakang	1
1.2. Tujuan Praktikum	1
1.3. Manfaat Praktikum.....	1
BAB II	2
PEMBAHASAN	2
2.1. Deskripsi Materi Single Linked List	2
2.2. NodeSLL_2511532005.....	2
2.3. TambahSLL_2511532005.....	3
2.4. PencarianSLL_2511532005.....	5
2.5. HapusSLL_2511532005.....	6
2.6. Analisis Praktikum.....	7
BAB III.....	8
KESIMPULAN DAN SARAN.....	8
3.1 Kesimpulan.....	8
3.2 Saran.....	8
DAFTAR PUSTAKA	9

BAB I

PENDAHULUAN

1.1. Latar Belakang

Struktur data merupakan cara untuk menyimpan dan mengelola data agar dapat digunakan secara efisien. Salah satu struktur data yang penting adalah **Single Linked List**, yaitu struktur data linear yang terdiri dari node-node yang saling terhubung. Single Linked List memiliki keunggulan dalam penggunaan memori yang dinamis serta fleksibilitas dalam operasi penambahan dan penghapusan data.

1.2. Tujuan Praktikum

1. Memahami konsep Single Linked List
2. Mengimplementasikan Single Linked List dalam Java
3. Memahami operasi dasar seperti insert, delete, dan search

1.3. Manfaat Praktikum

1. Melatih logika pemrograman
2. Memahami struktur data dinamis
3. Menjadi dasar untuk struktur data yang lebih kompleks

BAB II

PEMBAHASAN

2.1. Deskripsi Materi Single Linked List

Single Linked List merupakan salah satu struktur data linear yang terdiri dari kumpulan node yang saling terhubung. Setiap node memiliki dua komponen utama, yaitu data dan pointer (referensi) yang menunjuk ke node berikutnya. Struktur ini bersifat dinamis karena ukuran memori dapat berubah sesuai kebutuhan selama program berjalan. Berbeda dengan array, Single Linked List tidak memerlukan alokasi memori secara berurutan, sehingga lebih fleksibel dalam proses penambahan dan penghapusan data.

2.2. NodeSLL_2511532005

Class ini digunakan sebagai dasar dalam pembentukan struktur Single Linked List. Di dalamnya terdapat atribut untuk menyimpan data dan pointer ke node berikutnya. Class ini sangat penting karena menjadi elemen utama dalam linked list. Tanpa class node, struktur linked list tidak dapat dibentuk.

```
1 package Pekan5_2511532005;
2
3 public class NodeSLL_2511532005 {
4     // node bagian data
5     int data_2005;
6     // pointer ke node berikutnya
7     NodeSLL_2511532005 next_2005;
8     // konstruktor menginisialisasi node dengan data
9     public NodeSLL_2511532005(int data) {
10         this.data_2005 = data;
11         this.next_2005 = null;
12     }
13
14 }
15
```

Gambar 2.1. Kode program NodeSLL_2511532005.

Kode dan fungsi :

1. data_2005 berfungsi untuk menyimpan nilai dalam node
2. next_2005 berfungsi sebagai penunjuk ke node berikutnya.
3. this.data_2005 = data mengisi nilai node
4. this.next_2005 = null menandakan node belum terhubung ke node lain

2.3. TambahSLL_2511532005.

Class ini berfungsi untuk melakukan operasi penambahan node pada Single Linked List. Penambahan dapat dilakukan di berbagai posisi, seperti di depan, di akhir, maupun di posisi tertentu. Dengan adanya fungsi ini, linked list menjadi fleksibel dalam pengelolaan data. Selain itu, terdapat juga fungsi untuk menampilkan isi linked list.

```

1 package Pekan5_2511532005;
2
3 public class TambahSLL_2511532005 {
4     public static NodeSLL_2511532005 insertAtFront (NodeSLL_2511532005 head_2005, int value_2005) {
5         NodeSLL_2511532005 new_node_2005 = new NodeSLL_2511532005 (value_2005);
6         new_node_2005.next_2005 = head_2005;
7         return new_node_2005;
8     }
9     //fungsi menambahkan node di akhir SLL
10    public static NodeSLL_2511532005 insertAtEnd (NodeSLL_2511532005 head_2005, int value_2005) {
11        //buat sebuah node dengan sebuah nilai
12        NodeSLL_2511532005 newNode = new NodeSLL_2511532005 (value_2005);
13        //jika list kosong maka node jadi head
14        if (head_2005 == null) {
15            return newNode;
16        }
17        //simpan head ke variabel sementara
18        NodeSLL_2511532005 last = head_2005;
19        //telusuri ke node akhir
20        while (last.next_2005 != null) {
21            last = last.next_2005;
22        }
23        //ubah pointer
24        last.next_2005 = newNode;
25        return head_2005;
26    }
27    static NodeSLL_2511532005 GetNode (int data) {
28        return new NodeSLL_2511532005 (data);
29    }
30    static NodeSLL_2511532005 insertPos (NodeSLL_2511532005 headNode_2005, int position_2005, int value_2005) {
31        NodeSLL_2511532005 head_2005 = headNode_2005;
32        if (position_2005 < 1)
33            System.out.print ("Invalid position");
34        if (position_2005 == 1) {
35            NodeSLL_2511532005 new_node = new NodeSLL_2511532005 (value_2005);
36            new_node.next_2005 = head_2005;
37            return new_node;

```

```

380     } else {
390         while (position_2005-- != 0) {
400             if (position_2005 == 1) {
410                 NodeSLL_2511532005 newNode = GetNode (value_2005);
420                 newNode.next_2005 = headNode_2005.next_2005;
430                 headNode_2005.next_2005 = newNode;
440                 break;
450             }
460             headNode_2005 = headNode_2005.next_2005;
470         }
480         if (position_2005 != 1)
490             System.out.print ("Posisi di luar jangkauan");
500     }
510     return head_2005;
520 }
530 public static void printlist_2005 (NodeSLL_2511532005 head_2005) {
540     NodeSLL_2511532005 curr_2005 = head_2005;
550     while (curr_2005.next_2005 != null) {
560         System.out.print (curr_2005.data_2005+"->");
570         curr_2005 = curr_2005.next_2005;
580     }
590     if (curr_2005.next_2005 == null) {
600         System.out.print (curr_2005.data_2005);
610     }
620     System.out.println();
630 }
640 public static void main (String [] args) {
650     //buat linked list 2->3->5->6
660     NodeSLL_2511532005 head_2005 = new NodeSLL_2511532005 (2);
670     head_2005.next_2005 = new NodeSLL_2511532005 (3);
680     head_2005.next_2005.next_2005 = new NodeSLL_2511532005 (5);
690     head_2005.next_2005.next_2005.next_2005 = new NodeSLL_2511532005 (6);
700     //cetak list asli
710     System.out.print ("Senarai berantai awal: ");
720     printlist_2005 (head_2005);

```

```

73     //tambahkan node baru di depan
74     System.out.print ("tambah 1 simpul di depan: ");
75     int data_2005 = 1;
76     head_2005 = insertAtFront (head_2005, data_2005);
77     //cetak update list
78     printlist_2005 (head_2005);
79
80     //tambahkan node baru dibelakang
81     System.out.print ("tambah 1 simpul di belakang: ");
82     int data2_2005 = 7;
83     head_2005 = insertAtEnd (head_2005, data2_2005);
84
85     //cetak update list
86     printlist_2005 (head_2005);
87     System.out.print ("tambah 1 simpul ke data 4: ");
88     int data3_2005 = 4;
89     int pos_2005 = 4;
90     head_2005 = insertPos (head_2005, pos_2005, data3_2005);
91     //cetak update list
92     printlist_2005 (head_2005);
93
94
95 }
96 }

```

Gambar 2.2. Kode program TambahSLL_2511532005.

Kode dan fungsi :

1. insertAtFront() : Membuat node baru, menghubungkan node baru ke head lama dan menjadikan node baru sebagai head.
2. insertAtEnd() : Menelusuri hingga node terakhir dan menghubungkan node terakhir dengan node baru.
3. insertPos() : menyisipkan node pada posisi tertentu dan mengatur pointer node sebelumnya dan sesudahnya.
4. printlist_2005() : menampilkan semua data dalam linked list secara berurutan.

2.4. PencarianSLL_2511532005

Class ini digunakan untuk melakukan pencarian data dalam Single Linked List. Proses pencarian dilakukan dengan cara menelusuri node satu per satu dari head hingga node terakhir. Jika data yang dicari ditemukan, maka fungsi akan mengembalikan nilai true. Jika tidak ditemukan, maka akan mengembalikan false.

```

1 package Pekan5_2511532005;
2
3 public class PencarianSLL_2511532005 {
4     static boolean searchKey_2005(NodeSLL_2511532005 head_2005, int key_2005) {
5         NodeSLL_2511532005 curr_2005 = head_2005;
6         while (curr_2005 != null) {
7             if (curr_2005.data_2005 == key_2005)
8                 return true;
9             curr_2005 = curr_2005.next_2005; }
10        return false; }
11    public static void traversal_2005 (NodeSLL_2511532005 head_2005) {
12        // mulai dari head
13        NodeSLL_2511532005 curr_2005 = head_2005;
14        // telusuri sampai pointer null
15        while (curr_2005 != null) {
16            System.out.print(" " + curr_2005.data_2005);
17            curr_2005 = curr_2005.next_2005; }
18        System.out.println(); }
19
20    public static void main(String[] args) {
21        NodeSLL_2511532005 head_2005 = new NodeSLL_2511532005 (14);
22        head_2005.next_2005 = new NodeSLL_2511532005 (21);
23        head_2005.next_2005.next_2005 = new NodeSLL_2511532005 (13);
24        head_2005.next_2005.next_2005.next_2005 = new NodeSLL_2511532005 (30);
25        head_2005.next_2005.next_2005.next_2005.next_2005 = new NodeSLL_2511532005 (10);
26        System.out.print ("Penelusuran SLL: ");
27        traversal_2005 (head_2005);
28        //data yang akan dicari
29        int key_2005 = 30;
30        System.out.print ("cari data "+key_2005+ " = ");
31        if (searchKey_2005 (head_2005, key_2005))
32            System.out.println ("Ketemu");
33        else
34            System.out.println ("Tidak ada");
35    }
36
37
38
39

```

Gambar 2.3. Kode Program PencarianSLL_2511532005.

Kode dan fungsi :

1. searchKey_2005() : Melakukan traversal dari head membandingkan setiap data dengan nilai yang dicari, mengembalikan true jika ditemukan dan mengembalikan false jika tidak ditemukan.
2. transversal_2005 : Menampilkan seluruh isi linked list dan digunakan untuk membantu proses pengecekan data.

2.5. HapusSLL_2511532005

Class ini berfungsi untuk menghapus node dari Single Linked List. Penghapusan dapat dilakukan pada node pertama, node terakhir, maupun node pada posisi tertentu. Proses ini dilakukan dengan mengatur ulang pointer antar node. Dengan demikian, node yang dihapus tidak lagi terhubung dalam list.

```

1 package Pekan5_2511532005;
2
3 public class HapusSLL_2511532005 {
4     // fungsi untuk menghapus head
5     public static NodeSLL_2511532005 deleteHead_2005(NodeSLL_2511532005 head_2005) {
6         // jika SLL kosong
7         if (head_2005 == null)
8             return null;
9         // pindahkan head ke node berikutnya
10        head_2005 = head_2005.next_2005;
11        // return head baru
12        return head_2005;
13    }
14    // fungsi menghapus node terakhir SLL
15    public static NodeSLL_2511532005 removeLastNode(NodeSLL_2511532005 head_2005) {
16        // jika list kosong, return null
17        if (head_2005 == null) {
18            return null;
19        }
20        // jika list satu node, hapus node dan return null
21        if (head_2005.next_2005 == null) {
22            return null;
23        }
24        // temukan node terakhir ke dua
25        NodeSLL_2511532005 secondLast = head_2005;
26        while (secondLast.next_2005.next_2005 != null) {
27            secondLast = secondLast.next_2005;
28        }
29        // hapus node terakhir
30        secondLast.next_2005 = null;
31        return head_2005;
32    }
33    // fungsi menghapus node di posisi tertentu
34    public static NodeSLL_2511532005 deleteNode_2005(NodeSLL_2511532005 head_2005, int position_2005) {
35        NodeSLL_2511532005 temp_2005 = head_2005;
36        NodeSLL_2511532005 prev_2005 = null;
37        // jika linked list null
38        if (temp_2005 == null)
39            return head_2005;
40
41        // kasus 1 = head dihapus
42        if (position_2005 == 1) {
43            head_2005 = temp_2005.next_2005;
44            return head_2005;
45        }
46        // kasus 2 menghapus node di tengah
47        // telusuri ke node yang dihapus
48        for (int i = 1; temp_2005 != null && i < position_2005; i++) {
49            prev_2005 = temp_2005;
50            temp_2005 = temp_2005.next_2005;
51        }
52        // jika ditemukan, hapus node
53        if (temp_2005 != null) {
54            prev_2005.next_2005 = temp_2005.next_2005;
55        } else {
56            System.out.println(" Data tidak ada ");
57        }
58        return head_2005;
59    }
60    // fungsi mencetak SLL
61    public static void printList (NodeSLL_2511532005 head_2005) {
62        NodeSLL_2511532005 curr_2005 = head_2005;
63        while (curr_2005.next_2005 != null) {
64            System.out.print(curr_2005.data_2005+"-->");
65            curr_2005 = curr_2005.next_2005;
66        }
67        if (curr_2005.next_2005 == null) {
68            System.out.print(curr_2005.data_2005);
69        }
70        System.out.println();
71    }
72 }

```

```

71 // ===== MAIN =====
72 public static void main(String[] args) {
73
74     // buat SLL
75     NodeSLL_2511532005 head_2005 = new NodeSLL_2511532005(1);
76
77     head_2005.next_2005 = new NodeSLL_2511532005(2);
78     head_2005.next_2005.next_2005 = new NodeSLL_2511532005(3);
79     head_2005.next_2005.next_2005.next_2005 = new NodeSLL_2511532005(4);
80     head_2005.next_2005.next_2005.next_2005.next_2005 = new NodeSLL_2511532005(5);
81     head_2005.next_2005.next_2005.next_2005.next_2005.next_2005 = new NodeSLL_2511532005(6);
82
83     // cetak awal
84     System.out.println("List awal: ");
85     printList(head_2005);
86
87     // hapus head
88     head_2005 = deleteHead_2005(head_2005);
89     System.out.println("List setelah head dihapus: ");
90     printList(head_2005);
91
92     // hapus node terakhir
93     head_2005 = removeLastNode(head_2005);
94     System.out.println("List setelah simpul terakhir dihapus: ");
95     printList(head_2005);
96
97     // hapus posisi 2
98     int position_2005 = 2;
99     head_2005 = deleteNode_2005(head_2005, position_2005);
100
101     System.out.println("List setelah posisi 2 dihapus: ");
102     printList(head_2005);
103 }
104 }

```

Gambar 2.4. Kode program HapusSLL_2511532005.

Kode dan fungsi :

1. `deleteHead_2005()` : menggeser head ke node berikutnya dan node lama akan terhapus.
2. `removeLastNode()` : menelusuri hingga node kedua terakhir dan menghapus node terakhir dengan mengubah pointer.
3. `deleteNode_2005()` : menelusuri node sesuai posisi dan menghubungkan node sebelum dan sesudah node yang di hapus.
4. `printList()` : menampilkan isi linked list setelah penghapusan.

2.6. Analisis Praktikum.

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa implementasi Single Linked List memberikan pemahaman yang lebih dalam mengenai konsep struktur data dinamis. Operasi seperti penambahan, pencarian, dan penghapusan dapat dilakukan dengan efisien tanpa perlu menggeser data seperti pada array. Namun, proses traversal tetap diperlukan untuk mengakses data tertentu, sehingga membutuhkan waktu lebih lama dibandingkan struktur data dengan akses langsung. Secara keseluruhan, praktikum ini sangat membantu dalam memahami cara kerja pointer dan hubungan antar node dalam suatu struktur data.

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Single Linked List merupakan struktur data dinamis yang terdiri dari node-node yang saling terhubung melalui pointer. Struktur ini memungkinkan proses penambahan dan penghapusan data dilakukan dengan lebih fleksibel dibandingkan array. Melalui praktikum ini, telah dipahami berbagai operasi dasar seperti insert, delete, dan search pada linked list. Dengan demikian, pemahaman terhadap konsep dasar struktur data menjadi semakin meningkat.

3.2 Saran

Dalam mempelajari Single Linked List, disarankan untuk lebih sering berlatih dalam membuat dan memodifikasi program agar pemahaman semakin baik. Selain itu, perlu juga mempelajari variasi lain seperti Double Linked List dan Circular Linked List untuk memperluas wawasan. Penggunaan visualisasi seperti diagram node juga sangat membantu dalam memahami alur pointer. Terakhir, diharapkan praktikum selanjutnya dapat membahas struktur data yang lebih kompleks. (Linked list Data Structure)

DAFTAR PUSTAKA

- Java Program to create and display a singly linked list.* (n.d.). Retrieved from pointTech: <https://www.tpointtech.com/java-program-to-create-and-display-a-singly-linked-list>
- Linked list Data Structure.* (n.d.). Retrieved from Progamiz: <https://www.programiz.com/dsa/linked-list>

TUGAS PRAKTIKUM 5

Simulasi Antrian Rumah Sakit

1. Perintah Tugas

Pada tugas ini, mahasiswa diminta untuk mengimplementasikan konsep Single Linked List yang bekerja secara dinamis menggunakan pointer (referensi antar node). Tema tugas kali ini adalah Simulasi Antrian Rumah Sakit. Mahasiswa diwajibkan membuat program yang mensimulasikan sistem pendaftaran antrian pasien, di mana setiap pasien direpresentasikan sebagai sebuah node yang memiliki pointer ke pasien berikutnya. Pasien yang pertama kali mendaftar akan menjadi pasien pertama yang dipanggil (FIFO — First In, First Out).

2. ADT Pasien

```
1 package Pekan5_2511532005;
2
3 public class Pasien_2511532005 {
4     private String namaPasien_2005;
5     private String penyakit_2005;
6     private int nomorAntrian_2005;
7     private Pasien_2511532005 next_2005;
8
9     // inisialisasi atribut
10    public Pasien_2511532005(String namaPasien_2005, String penyakit_2005, int nomorAntrian_2005) {
11        this.namaPasien_2005 = namaPasien_2005;
12        this.penakit_2005 = penyakit_2005;
13        this.nomorAntrian_2005 = nomorAntrian_2005;
14        this.next_2005 = null;
15    }
16    // Getter
17    public String getNamaPasien_2005() { return namaPasien_2005; }
18    public String getPenyakit_2005() { return penyakit_2005; }
19    public int getNomorAntrian_2005() { return nomorAntrian_2005; }
20    public Pasien_2511532005 getNext_2005() { return next_2005; }
21
22    // setter
23    public void setNamaPasien_2005(String namaPasien_2005) {
24        this.namaPasien_2005 = namaPasien_2005;
25    }
26    public void setPenyakit_2005(String penyakit_2005) {
27        this.penakit_2005 = penyakit_2005;
28    }
29    public void setNext_2005(Pasien_2511532005 next_2005) {
30        this.next_2005 = next_2005;
31    }
32    public void setNext_2005(Pasien_2511532005 next_2005) {
33        this.next_2005 = next_2005;
34    }
35 }
36
```

3. Drive Rumah Sakit

```
1 package Pekan5_2511532005;
2
3 import java.util.Scanner;
4
5 public class RumahSakit_2511532005 {
6     private Pasien_2511532005 head_2005;
7     private int counter_2005;
8
9     public RumahSakit_2511532005() {
10         this.head_2005 = null;
11         this.counter_2005 = 0;
12     }
13
14     // Daftarkan Pasien (Insert at Tail) - Auto increment nomor antrian
15     public void daftarkanPasien_2005(String namaPasien_2005, String penyakit_2005) {
16         counter_2005++;
17         Pasien_2511532005 newPasien_2005 = new Pasien_2511532005(namaPasien_2005, penyakit_2005, counter_2005);
18
19         if (head_2005 == null) {
20             // Jika list kosong, node baru jadi head
21             head_2005 = newPasien_2005;
22         } else {
23             // Telusuri ke node terakhir
24             Pasien_2511532005 curr_2005 = head_2005;
25             while (curr_2005.getNext_2005() != null) {
26                 curr_2005 = curr_2005.getNext_2005();
27             }
28             // Set pointer next node terakhir ke node baru
29             curr_2005.setNext_2005(newPasien_2005);
30         }
31         System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " + counter_2005);
32     }
33
34     // Panggil Pasien (Delete Head)
35     public void panggilPasien_2005() {
36         if (head_2005 == null) {
37             System.out.println("Antrian kosong, tidak ada pasien yang dapat dipanggil.");
38         } else {
39             System.out.println("Pasien yang dipanggil:");
40             System.out.println("Nomor Antrian: " + head_2005.getNomorAntrian_2005());
41             System.out.println("Nama : " + head_2005.getNamaPasien_2005());
42             System.out.println("Keluhan : " + head_2005.getPenyakit_2005());
43             // Geser head ke node berikutnya
44             head_2005 = head_2005.getNext_2005();
45         }
46     }
47
48     // Tampilkan Antrian (Display) - Menampilkan posisi pasien
49     public void tampilkanAntrian_2005() {
50         if (head_2005 == null) {
51             System.out.println("Antrian kosong.");
52             return;
53         }
54         Pasien_2511532005 curr_2005 = head_2005;
55         int posisi_2005 = 1;
56         System.out.println("Daftar Antrian Pasien:");
57         while (curr_2005 != null) {
58             System.out.println("Posisi " + posisi_2005 + " - Nomor Antrian: " +
59                 curr_2005.getNomorAntrian_2005() +
60                 ", Nama: " + curr_2005.getNamaPasien_2005() +
61                 ", Keluhan: " + curr_2005.getPenyakit_2005());
62             curr_2005 = curr_2005.getNext_2005();
63             posisi_2005++;
64         }
65     }
66 }
```

```

67 // Cari Pasien (Search) - Case insensitive
68 public void cariPasien_2005(String namaCari_2005) {
69     if (head_2005 == null) {
70         System.out.println("Antrian kosong.");
71         return;
72     }
73     Pasien_2511532005 curr_2005 = head_2005;
74     boolean found_2005 = false;
75     while (curr_2005 != null) {
76         if (curr_2005.getNamaPasien_2005().equalsIgnoreCase(namaCari_2005)) {
77             System.out.println("Pasien ditemukan:");
78             System.out.println("Nomor Antrian: " + curr_2005.getNomorAntrian_2005());
79             System.out.println("Nama      : " + curr_2005.getNamaPasien_2005());
80             System.out.println("Keluhan   : " + curr_2005.getPenyakit_2005());
81             found_2005 = true;
82             break;
83         }
84         curr_2005 = curr_2005.getNext_2005();
85     }
86     if (!found_2005) {
87         System.out.println("Pasien dengan nama '" + namaCari_2005 + "' tidak ditemukan.");
88     }
89 }
90
91 // Cek Status Antrian
92 public void cekStatusAntrian_2005() {
93     if (head_2005 == null) {
94         System.out.println("Antrian kosong.");
95         return;
96     }
97     int totalPasien_2005 = 0;
98     Pasien_2511532005 curr_2005 = head_2005;
99     while (curr_2005 != null) {
100         totalPasien_2005++;
101         curr_2005 = curr_2005.getNext_2005();
102     }

```

```

103     System.out.println("Total pasien dalam antrian: " + totalPasien_2005);
104     System.out.println("Pasien terdepan:");
105     System.out.println("Nomor Antrian: " + head_2005.getNomorAntrian_2005());
106     System.out.println("Nama      : " + head_2005.getNamaPasien_2005());
107     System.out.println("Keluhan   : " + head_2005.getPenyakit_2005());
108 }
109
110 // Main method dengan menu sesuai contoh output
111 public static void main(String[] args) {
112     RumahSakit_2511532005 rs_2005 = new RumahSakit_2511532005();
113     Scanner input_2005 = new Scanner(System.in);
114     int pilihan_2005;
115
116     do {
117         System.out.println("\n=== Antrian Rumah Sakit NIM: 2511532005 ===");
118         System.out.println("1. Daftarkan Pasien (Insert)");
119         System.out.println("2. Panggil Pasien (Delete Head)");
120         System.out.println("3. Tampilkan Antrian (Display)");
121         System.out.println("4. Cari Pasien (Search)");
122         System.out.println("5. Cek Status Antrian");
123         System.out.println("6. Keluar");
124         System.out.print("Pilihan: ");
125
126         pilihan_2005 = input_2005.nextInt();
127         input_2005.nextLine();
128
129         switch (pilihan_2005) {
130             case 1:
131                 System.out.print("Masukkan Nama Pasien : ");
132                 String nama_2005 = input_2005.nextLine();
133                 System.out.print("Masukkan Keluhan : ");
134                 String keluhan_2005 = input_2005.nextLine();
135                 rs_2005.daftarkanPasien_2005(nama_2005, keluhan_2005);
136                 break;
137             case 2:
138                 rs_2005.panggilPasien_2005();
139                 break;

```

```

140 case 3:
141     rs_2005.tampilkanAntrian_2005();
142     break;
143 case 4:
144     System.out.print("Masukkan Nama Pasien yang dicari : ");
145     String cari_2005 = input_2005.nextLine();
146     rs_2005.cariPasien_2005(cari_2005);
147     break;
148 case 5:
149     rs_2005.cekStatusAntrian_2005();
150     break;
151 case 6:
152     System.out.println("Terima kasih!");
153     break;
154 default:
155     System.out.println("Pilihan tidak valid. Silakan coba lagi.");
156 }
157 } while (pilihan_2005 != 6);
158
159 input_2005.close();
160 }
161 }
162

```